Claude SDK for Pharo:
Pharo-Specific Implementation Notes

Punt Labs

April 2026 — v0.3.0-dev

Contents

1 Relationship to Go SDK **2**

2 Go to Smalltalk Mapping **2**

3 JSON Key Mapping **2**

4 Helper Constructors **2**

5 JSON Serialization **3**
5.1 Conventions . . . 3

6 Pharo Threading Model (Streaming) **3**

7 Porting from Go SDK **4**

8 Postern Bootstrap **4**

9 Agent-Assisted Development **4**

10 Package Health **5**

11 Package Architecture **5**
11.1 Package Inventory . . . 5
11.2 Dependency Flow . . . 5

12 Concurrency Model **6**
12.1 The Go SDK is Synchronous . . . 6
12.2 What Runs Where . . . 6
12.3 Safe Patterns . . . 6
12.4 Unsafe Patterns . . . 6

13 Smalltalk Idioms **6**
13.1 Composed Method (Pattern 1) . . . 7
13.2 Intention Revealing Selector (Pattern 2) . . . 7
13.3 Pluggable Behavior (Pattern 13) . . . 7
13.4 Collecting Parameter (Pattern 14) . . . 7
13.5 Method Object (Pattern 15) . . . 7

14 Go SDK Type Inventory (Condensed) **7**

15 Pharo Threading Quick Reference **8**

About this Document

Pharo-specific notes without an Anthropic counterpart. Companion to *claude-sdk-specification.tex*.

1 Relationship to Go SDK

The Go SDK (`anthropic-sdk-go`) is code-generated from Anthropic's OpenAPI specification via Stainless. This produces 232 types in `message.go` alone, most of which are mechanical variants of a smaller set of core concepts.

The Smalltalk SDK does not replicate this generated structure. Instead, it distills the Go types into a smaller class hierarchy organized by role, using Smalltalk's polymorphic dispatch and class hierarchy in place of Go's discriminated unions and struct embedding.

2 Go to Smalltalk Mapping

The Go SDK uses flat structs with discriminator fields and `AsAny()` type switches for polymorphism. Smalltalk replaces this with a class hierarchy and polymorphic message dispatch.

Go Pattern	Smalltalk Equivalent	Example
Struct with fields	Class with instance variables	`ClaudeMessage`
Discriminated union (`AsAny()`)	Abstract class + subclasses	`ClaudeContentBlock` hierarchy
`param.Opt[T]` (optional)	`nil` or value	Standard Smalltalk
`json.RawMessage`	Raw JSON string	String or Dictionary
Param vs Response types	Single class (Smalltalk is dynamic)	`ClaudeMessage` for both
Constants (`type X string`)	Class-side methods returning symbols	`ClaudeModel class>>sonnet`

3 JSON Key Mapping

Wire keys are snake_case strings (`max_tokens`). Smalltalk instance variables are camelCase (`maxTokens`). Every type class defines a class-side method `#jsonKeyMap` returning a Dictionary mapping instance variable names (Symbols) to wire key strings:

JSON key mapping

```
ClaudeMessage class>>jsonKeyMap
    ^ {
        #stopReason -> 'stop_reason'.
        #stopSequence -> 'stop_sequence'.
        #inputTokens -> 'input_tokens'.
    } asDictionary
```

The `#asJson` method iterates instance variables, looks up the wire key from the map (defaulting to the variable name if absent), and omits nil values. The class-side `#fromJson:` reverses the map to populate instance variables from the parsed Dictionary.

4 Helper Constructors

Class-side factory methods on content block classes provide concise construction:

Helper constructors

```
ClaudeTextBlock text: 'Hello, world'.
ClaudeImageBlock base64: encodedData mediaType: 'image/png'.
ClaudeToolUseBlock id: 'tu_123' name: 'get_weather' input: inputDict.
ClaudeToolResultBlock toolUseId: 'tu_123' content: 'Sunny, 22C'.
ClaudeToolResultBlock toolUseId: 'tu_123' content: 'Not found' isError: true.
ClaudeThinkingBlock thinking: 'Let me reason...' signature: 'sig...'.
```

```
ClaudeRedactedThinkingBlock data: 'redacted-data-string'.
ClaudeDocumentBlock fromBase64: 'JVBERi0xLjQ='.
ClaudeDocumentBlock fromFileId: 'file-abc123'.
ClaudeDocumentBlock fromText: 'Plain text document content'.
ClaudeDocumentBlock fromUrl: 'https://example.com/doc.pdf'.
ClaudeImageBlock fromUrl: 'https://example.com/image.png'.
ClaudeImageBlock fromFileId: 'file-img456'.
```

5 JSON Serialization

The Go SDK uses a custom JSON codec (`internal/apijson/`) with reflection-based encoder/decoder caches. Smalltalk does not need this — STONJSON handles serialization directly.

5.1 Conventions

- **Request serialization:** Each type implements `#asJson` returning a Dictionary. `STONJSON toString:` converts to wire format.
- **Response deserialization:** `STONJSON fromString:` returns a Dictionary (specifically `OrderedDictionary` in Pharo 12). Each type's class-side `#fromJson:` constructs typed objects.
- **Nil omission:** `#asJson` omits nil-valued fields. The Go SDK uses `param.Opt[T]` for this; Smalltalk uses nil naturally.
- **Key format:** Wire keys are snake_case strings (`max_tokens`). Smalltalk instance variables are camelCase (`maxTokens`). The `#jsonKeyMap` class-side method defines the mapping (see the JSON Key Mapping section above).
- **Unknown fields:** When deserializing, unknown JSON keys are silently ignored. This provides forward compatibility when the API adds new fields.

6 Pharo Threading Model (Streaming)

Pharo uses cooperative scheduling within a priority level. `[ ... ] fork` spawns a `Process` (green thread) that runs cooperatively with other processes at the same priority. Processes at the same priority do *not* run in parallel — they yield at I/O points, explicit yields, and semaphore waits. Processes at different priorities are preemptive (higher priority preempts lower).

Rule: streaming network I/O must run in a forked Process.

Simple streaming pattern (buffered)

```
| events sem |
events := OrderedCollection new.
sem := Semaphore new.

[ [ client
    streamMessage: params
    do: [ :event | events add: event ].
  sem signal ]
    on: ClaudeApiError
    do: [ :e | events add: e. sem signal ] ]
  forkAt: Processor userBackgroundPriority
  named: 'Claude stream'.

"Consumer waits for completion, then processes all events:"
sem wait.
events do: [ :each |
    each isKindOf: ClaudeApiError
        ifTrue: [ self handleError: each ]
        ifFalse: [ self handleEvent: each ] ]
```

The SDK provides `#streamMessage:do:` as a synchronous method. The consumer is responsible for forking. The SDK documents this requirement but does not enforce it — forcing a fork inside the SDK would prevent consumers from choosing their own concurrency strategy.

For incremental delivery, use the real-time mode pattern (SharedQueue + forked socket reader) described in the main specification's Streaming Messages section.

7 Porting from Go SDK

The reference implementation is Anthropic's Go SDK (`anthropic-sdk-go`), code-generated from the OpenAPI specification via Stainless. The Go SDK produces 232 types in `message.go` alone — most are mechanical variants of a smaller set of core concepts (param vs. response structs, union wrappers, JSON marshalers).

The porting strategy is *distillation, not transliteration.* Smalltalk's class hierarchy and polymorphic dispatch replace Go's flat structs and discriminated unions. A single `ClaudeContentBlock` hierarchy with 16 subclasses replaces 30+ Go `ContentBlock*` types. `ClaudeMessage` serves as both request parameter and response — Go needs separate `Message` and `MessageParam` structs because the type system cannot share them.

Each Go type is mapped once via the type inventory (see the Go SDK Type Inventory section below). When a Go type group collapses to a single Smalltalk class, the mapping is recorded and the redundant Go variants are not ported. This keeps the Smalltalk class count proportional to the number of *concepts*, not the number of generated types.

The reference SDK is kept at `.tmp/anthropic-sdk-go/` (gitignored) and consulted for every new API feature to ensure wire-format compatibility.

8 Postern Bootstrap

All development happens inside a live Pharo 12 image via Postern, an HTTP eval server running on `localhost:${EVAL_PORT:-8422}`. The eval server accepts Smalltalk expressions as plain-text POST bodies and returns printed results. This provides interactive class definition, method compilation, test execution, lint checks, and full introspection — all driven from Claude Code's Bash tool or any HTTP client.

The image is disposable. `make rebuild` creates a fresh image from scratch, loading all source from Tonel files in `src/`. `make stop` kills without saving — saving would capture socket state that causes errors on restore. The source of truth is always Tonel on disk, committed via Iceberg (libgit2) from inside the image.

Key Postern targets: `make setup` (download Pharo 12 + load packages), `make start` (launch with eval server), `make test` (run SDK test suite), `make lint` (Renraku lint on all classes), `make drift` (compare in-image methods vs. on-disk Tonel), `make filein` (reload Tonel into running image).

Multiple images can run simultaneously on different ports via the `EVAL_PORT` environment variable, each with its own PID file and log file.

9 Agent-Assisted Development

Development uses ethos mission pipelines to delegate work to specialist agents. The standard pipeline has five stages: design, implement, test, review, document. Each stage produces an artifact that feeds the next via `inputs_from` dependency wiring.

Implementation work is dispatched to a Smalltalk specialist agent (kwb) scoped by a mission contract that specifies the write-set (which Tonel files may be touched), success criteria (`make lint` clean, specific test classes passing, Iceberg ref sync), round budget, and evaluator. Every contract is peer-reviewed by an architect agent before the worker sees it.

The SDK's own `ClaudeExchange` agent runtime is used during development — the implementing agent runs inside the Pharo image via the eval server, compiling methods and running tests through the same SDK classes it is building. This in-image dogfooding exercises interaction patterns that no external test harness can simulate: streaming under load, tool dispatch cycles, error recovery across conversation turns.

Quality gates are layered: the worker runs per-class tests via the eval server, the leader runs `make lint` from CLI (which catches class-level Renraku rules that per-method probes miss), and local review agents scan the full branch diff before any PR is created.

10 Package Health

Package structure is assessed using the Ducasse/Abdeen modularization metrics from WCRE'11 (*Package Fingerprints*). The key metric is IIPU (Index of Inter-Package Usage) — the ratio of intra-package references to total references. Higher IIPU means better encapsulation.

The SDK tracks IIPU across restructuring milestones. The original flat `Claude-SDK-*` layout scored IIPU 0.458 (heavy cross-package coupling). After the Messaging/Agent layer split and the introduction of `Claude-Messaging-Tools` and `Claude-Messaging-Files` as separate packages, IIPU rose to 0.838. The `/package-health` assessment skill provides repeatable analysis.

Naming discipline follows Kent Beck's Intention Revealing Selector pattern, scaled from methods to packages. Package names follow the Pharo idiom `Family-SubFamily-Module` (e.g., `Claude-Messaging-Types`, `Claude-Agent-Tools-Pharo`). The layered names make the dependency hierarchy explicit: everything in `Claude-Agent-*` depends on `Claude-Messaging-*`, never the reverse. Tool names use PascalCase compound names matching Claude Code's convention (`BrowseClass`, `CompileMethod`, `RunTests`).

Every package name change and every new package is reviewed by an architect agent for consistency with the naming scheme before implementation begins.

11 Package Architecture

11.1 Package Inventory

Package	Classes	Responsibility
`Claude-Messaging-Types`	48	`ClaudeMessage` and decomposition (turn, stop details, request context, container), content block hierarchy (16 subclasses including `ClaudeDocumentBlock` with source hierarchy), tool types, model, usage, cache control, thinking params, sampling params, request options, output format; JSON serialization; helper constructors
`Claude-Messaging-Errors`	10	`ClaudeApiError` hierarchy (8 concrete subclasses keyed on HTTP status), error factory, `ClaudeInvalidRequestError`
`Claude-Messaging-Streaming`	2	`ClaudeSSEDecoder` (buffered and real-time SSE parsing), `ClaudeStreamingSocket` (raw TLS with chunked transfer-encoding)
`Claude-Messaging-Client`	1	`ClaudeClient` — creation, configuration, HTTP request lifecycle, options, retry logic, Files API
`Claude-Messaging-Tools`	10	`ClaudeServerTool` hierarchy (8 concrete server-side tools), name-keyed `TypeRegistry`
`Claude-Messaging-Files`	4	`ClaudeFileMetadata`, `ClaudeFilePage`, `ClaudeFileListParams`, `ClaudeDeletedFile`
`Claude-Messaging-Examples`	7	Example scripts for messaging API usage

11.2 Dependency Flow

Dependencies flow in strict layers:

```
  Messaging-Types  Messaging-Errors  Messaging-Streaming
        |                |                 |
  Messaging-Tools  Messaging-Files        |
        |                |                 |
        +-------+-------+-------+--------+
                |
          Messaging-Client
```

Messaging-Client depends on Types, Errors, Streaming, Tools (server-side tool types), and Files (file metadata types).

The Agent layer (`Claude-Agent-*`) depends on Messaging-Client and is specified in the *Claude Agent SDK for Pharo — Technical Specification*.

12 Concurrency Model

12.1 The Go SDK is Synchronous

The Go SDK has no goroutines in its public API. All methods are blocking. `http.Client` manages connection pooling internally. The Smalltalk SDK mirrors this: all public methods are synchronous and blocking.

12.2 What Runs Where

Operation	Blocking?	Safe for UI?
Create client	No	Yes
Build message params	No	Yes
`#sendMessage:`	Yes (network I/O)	**No — fork**
`#streamMessage:do:`	Yes (network I/O)	**No — fork**
`#listModels`	Yes (network I/O)	**No — fork**
`#getModel:`	Yes (network I/O)	**No — fork**
`#countTokens:`	Yes (network I/O)	**No — fork**
Parse JSON response	No	Yes
Tool handler execution	Depends	Fork if I/O-bound
Retry backoff sleep	Yes	**No — fork**

12.3 Safe Patterns

Fork + Semaphore
: Fork a Process, signal a Semaphore on completion. Consumer waits on the Semaphore. Use when the consumer needs the full result before proceeding.

Fork + SharedQueue
: Fork a Process, enqueue results into a SharedQueue. Consumer drains the queue incrementally. Use for streaming events where partial results are useful. The consumer reads from the queue in a loop, not after a semaphore wait.

Fork + callback
: Fork a Process, call a block on each event. The block must not touch Morphic directly — use `WorldState defer:` to schedule UI updates on the UI thread.

12.4 Unsafe Patterns

- Calling `#sendMessage:` directly from a Morphic event handler.
- Calling `#streamMessage:do:` without forking.
- Modifying Morphic submorphs from a background Process without `WorldState defer:`.
- Waiting on a Semaphore from a SharedQueue-based streaming pattern (blocks until complete, defeating incremental delivery).

13 Smalltalk Idioms

Kent Beck's *Smalltalk Best Practice Patterns* guide the SDK design.

13.1 Composed Method (Pattern 1)

Every method does one identifiable thing. Large methods are decomposed into helper messages. The Go SDK's `ExecuteNewRequest` (80+ lines) becomes several composed methods in Smalltalk: `#buildRequest:`, `#executeWithRetry:`, `#parseResponse:`.

13.2 Intention Revealing Selector (Pattern 2)

Method names describe *what*, not *how*:

- `#sendMessage:` not `#postToMessagesEndpoint:`
- `#textContent` not `#collectTextBlocksAndJoin`
- `#toolUseBlocks` not `#selectContentBlocksOfTypeToolUse`

13.3 Pluggable Behavior (Pattern 13)

Use blocks for extensibility:

- `#streamMessage:do:` takes a callback block.
- Tool runner handlers are blocks.
- Retry policy could be a pluggable strategy block.

13.4 Collecting Parameter (Pattern 14)

Build results incrementally. The message accumulator is a collecting parameter: it receives streaming events one at a time and constructs the full message progressively.

13.5 Method Object (Pattern 15)

When a method gets too complex, extract it into a separate class whose instances represent a single execution of the algorithm. In this SDK, the following are already designed as method objects (separate classes, not methods buried in `ClaudeClient`):

- `ClaudeSSEDecoder` — encapsulates SSE parsing state
- `ClaudeExchange` — encapsulates the tool dispatch loop with iteration count and state inspection (see the *Claude Agent SDK Specification*)

The retry loop is part of `ClaudeClient`'s request lifecycle. If retry logic grows more complex (e.g., pluggable retry policies), it should be extracted into a `ClaudeRetryPolicy` method object.

14 Go SDK Type Inventory (Condensed)

The Go SDK's `message.go` contains 232 types. Most are generated variants. The following table maps every Go type group to the Smalltalk class that replaces it.

Go Type Group	Smalltalk Class	Notes
Message, MessageJSON	`ClaudeMessage`	Response + param; includes `type` field
MessageParam	via `#asParam`	Converter method
MessageNewParams	`ClaudeMessageRequest`	Request builder
ContentBlockUnion (12+ variants)	`ClaudeContentBlock` subclasses	Polymorphic hierarchy
ContentBlockParamUnion	Same classes, `#asJson`	Shared
TextBlock (with Citations)	`ClaudeTextBlock`	`text` + `citations`
ThinkingBlock	`ClaudeThinkingBlock`	`thinking`, `signature`
RedactedThinkingBlock	`ClaudeRedactedThinkingBlock`	`data`

Go Type Group	Smalltalk Class	Notes
ServerToolUse, WebSearch*, etc.	Server result subclasses	Read-only, raw JSON stored
Usage, MessageDeltaUsage	`ClaudeUsage`	Single class
ToolParam, ToolUnionParam	`ClaudeTool`	Unified
ToolInputSchemaParam	`ClaudeToolInputSchema`	Schema builder
ToolResultBlockParam	`ClaudeToolResultBlock`	In hierarchy
ToolChoiceUnionParam	Dictionary	Dynamic structure
ThinkingConfigParamUnion	`ClaudeThinkingParams`	Typed class; 3 modes: enabled, adaptive, disabled
StopReason (enum)	Symbol	6 values
Model (enum)	`ClaudeModel` class methods	Factory pattern; alias + dated
ModelInfo	`ClaudeModelInfo`	From Models API
MessageTokensCount	`ClaudeMessageTokensCount`	From CountTokens
MessageStreamEventUnion	`ClaudeStreamEvent` subclasses	Polymorphic
*Delta types (5)	Embedded in event subclasses	Simplified
RawContentBlockDeltaUnion	Discriminated in accumulator	5 delta subtypes
Base64PDFSource	`ClaudeDocumentBlock`	Document content block with `ClaudeDocumentSource` hierarchy (5 source types)
CacheControlEphemeralParam	`ClaudeCacheControl`	Typed class with optional TTL
MetadataParam	Dictionary	Dynamic structure
ServiceTier	String	`'standard'` etc.
MessageBatch*	Deferred	v2
Beta*	Deferred	v2
Bedrock*, Vertex*	Out of scope	—
Completion*	Out of scope	Legacy

15 Pharo Threading Quick Reference

Pharo uses cooperative scheduling. Processes at the same priority level run one at a time, yielding at I/O waits, explicit yields (`Processor yield`), and semaphore waits. Higher-priority processes preempt lower ones. This is *not* parallel execution — a CPU-bound process at user priority will starve other user-priority processes until it yields.

Primitive	Usage
`[ ... ] fork`	Spawn a Process at the same priority as the caller. Does not inherit the caller's stack.
`[ ... ] forkAt:named:`	Spawn at a specific priority with a name. Use `Processor userBackgroundPriority` for SDK work.
`Semaphore new`	Binary signal/wait. Use for "notify when done."
`SharedQueue new`	Thread-safe FIFO. Use for producer/consumer event passing. `next` blocks until an item is available.
`Mutex new`	Mutual exclusion. Use for shared mutable state (rare in this SDK).
`Delay forSeconds:`	Non-busy wait. Yields to other processes during the delay. Used for retry backoff.
`WorldState defer: [ ... ]`	Schedule a block to run on the Morphic UI thread. Required for any Morph modification from a background Process.
`Process>>terminate`	Kill a running process. Use for cancellation (e.g., abort a streaming request). Closes any associated sockets.